

Elliptic bridges and bounded saturation

Point laws, typed packets and explicit research boundaries

Kernel-checked generalized addition and binary multiplication, tripling identities, typed root and fibre semantics, bounded repeated prime preimages and real Legendre finite-part enclosures.

5

new formal modules

6

worked saturation packets

5

real endpoint packets

Specified formal interfaces and bounded arithmetic. Global Mordell bounds, Matveev, full JSON/compiler semantics and arbitrary singular periods remain open.

Source, corpus receipts and reproduction commands accompany the repository snapshot.

This release proves five focused formal interfaces, supplies bounded repeated rational prime preimages with witnessed generator compression, and certifies the real Legendre nodal finite part by exact rational enclosures. It integrates concurrent original-model halving transport, literal-list and signed-lift refinements, as well as flavor functional-mass, operator-mixing and dimensionful-wall work. It does not prove a general global Mordell bound, remove Matveev premises, formalize the entire query compiler, or certify arbitrary singular period matrices.

Generalized point-law refinement

Write a generalized Weierstrass curve as $y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$. Encode infinity by none and a nonsingular affine point by some(x,y). The typed coordinate algorithm retains both identity branches and the vertical condition $x=u$ and $y=-v-a_1u-a_3$. Away from that condition its slope is the secant $(y-v)/(x-u)$ if x differs from u , or the tangent $(3x^2 + 2a_2x + a_4 - a_1y)/(2y + a_1x + a_3)$ otherwise. Its output has $z=s^2 + a_1s - a_2 - x - u$ and ordinate $-(s(z-x)+y) - a_1z - a_3$.

EllipticPointLaw.add_refinement proves that these coordinates equal the coordinates of the sum in Mathlib's actual point group. The proof covers infinity, secants, tangents and vertical sums by the exact branches of the native point law. It is a generic field theorem and requires nonsingularity through the input point type. The coordinate program can be applied to untyped pairs, but the theorem makes no claim for such pairs.

The binary interpreter reads a little-endian Boolean list with value $b_0 + 2b_1 + 4b_2 + \dots$. It doubles the recursively interpreted tail, then adds the source if the current bit is one. binary_refinement proves its output equals the actual natural scalar multiple for every bit list, including the empty zero list. This establishes the mathematical binary algorithm. Python's JSON parsing, integer-to-bit conversion, allocation limits and machine execution are separate interfaces.

Tripling identities

On the short model $y^2 = x^3 + ax + b$ put $f = x^3 + ax + b$, $d = 3x^2 + a$, $\psi = 3x^4 + 6ax^2 + 12bx - a^2$, and $F6 = x^6 + 5ax^4 + 20bx^3 - 5a^2x^2 - 4abx - 8b^2 - a^3$. The numerator $\phi = x\psi^2 - 8fF6$ is the producer's compact expression. expanded_phi proves its equality with the replay expression $x^9 - 12ax^7 - 96bx^6 + 30a^2x^5 - 24abx^4 + (36a^3 + 48b^2)x^3 + 48a^2bx^2 + (9a^4 + 96ab^2)x + 8a^3b + 64b^3$.

Two further identities are $12xf - d^2 = \psi$ and $(d\psi - 16f^2)^2 - d^2\psi^2 + 4fx\psi^2 = 4f\phi$. They hold by polynomial normalization over every field. The second is the denominator-free algebra behind a tangent followed by a secant. For ψ nonzero, tripling_x_iff identifies $\phi/\psi^2 = u$ with $\phi - u\psi^2 = 0$. Kernel and branch cases are handled by the existing fibre algorithms rather than assigned affine quotients. A single generic theorem identifying this quotient with the actual triple point in every exceptional case is not added here; the polynomial identities alone should not be advertised as that stronger theorem.

Short translation uses $X = x + A/3$, $a = B - A^2/3$ and $b = C - AB/3 + 2A^3/27$, with the target shifted by $A/3$.

Typed roots and staged fibres

A RationalPacket stores integral coefficients, a nonzero rational scale, the native coefficient validity proof, monicity, and the exact scaleRoots polynomial identity. Its interpreter is the proved native signed-divisor root generator. Its complete theorem derives rational-root completeness from these checkable data. The producer cannot place an arbitrary root-completeness assertion inside this packet. native_rational_certificate now emits this typed object and its theorem in addition to the existing original-source theorem; the worked polynomial $x^2-3x/2+1/2$ has exactly the roots $1/2$ and 1 , checked by the Lean kernel.

A FibrePacket has an actual group target, an integer multiplier, a list of actual group points, and a proof that list membership is equivalent to the multiplication equation. Its composition flattens the complete inner fibre over every attached outer point. If $nH=P$ and $mQ=H$, then $nmQ=P$. Conversely a point satisfying $nmQ=P$ has the forced intermediate $H=mQ$; outer completeness retains H and inner completeness retains Q . compose_complete proves both directions without any search bound, and includes empty fibres and duplicate list entries. Proof fields are obligations for the native emitter, not declarations that JSON replay already constitutes a kernel proof.

The rational-root producer is connected to its typed interpreter. Concurrent work closed original-model additive transport, exact literal halving-list equality and complete signed quartic lifts, including nonsquare obstructions. This release preserves those proofs and emits both completed-model and original-model typed fibre objects from all native halving routes. A worked native two-stage packet proves the complete rational four-division fibre over $4(3,5)$ on $y^2=x^3-2$ by composing two such objects. Automatically constructing native typed packets from every staged two/three packet, especially tripling and mixed branches, remains open. No JSON parser implementation proof is claimed.

Bounded repeated saturation

For a supplied subgroup G generated by at most four rational points, the established single-prime operation computes all generators of $D_p(G)=\{H:pH \text{ belongs to } G\}$, for $p=2$ or 3 . Every projective coefficient line in F_p^r is tested with its complete rational division fibre, and the rational kernel is retained. Dependent generators are allowed. This release repeats that complete operation and compresses its finite presentation using signed integer relations.

To remove a current generator U , the packet supplies integer coefficients expressing U as a combination of the other current generators. Each removal is replayed against the current list at that exact stage. The subgroup is unchanged. The producer searches a bounded coefficient box; failure, an oversized box or a rational-coordinate resource limit gives an unresolved membership question, not a proof that U is independent or that an extension has strict index.

Each prime stage carries the whole single-prime packet, including all residue combinations. To certify closure it supplies a combination of the previous generators for

every raw generator of $\text{Dp}(G)$. The single-prime generation theorem supplies G contained in $\text{Dp}(G)$, and these witnesses supply the reverse inclusion. Thus $\text{Dp}(G)=G$. Previously certified primes survive a stage only when this equality is proved. An unresolved stage clears the closure set and restarts the prime schedule. This avoids declaring joint saturation after an enlargement that might expose a previously hidden division relation.

The stopping states are closed, step-limit and width-limit. Closed means equality is witnessed for every requested prime on the final subgroup. Step-limit returns the exact final completed preimage group after the requested number of stages. Width-limit returns the exact final preimage group when compression has not reduced the next input to four generators. Neither limit state asserts saturation. Root nodes are shared across the request; exceeding the root budget raises a resource failure rather than emitting a partial fibre as complete.

The independent checker does no membership search. It replays the complete fibres, checks every signed compression relation and closure equation, checks the source-to-stage links, reconstructs the prime-reset schedule, and binds the final status and generators. It has its own packet and arithmetic work limits. The Python interpreter and the generic subgroup-generation proof remain separate from a complete Lean verification of this driver.

The rank-one witness starts at $6(3,5)$ on $y^2=x^3-2$. Five stages, at primes $2,2,3,2,3$, recover $(3,5)$ and certify joint closure. The $y^2=x^3+1$ example starts with the zero subgroup, acquires the rational two- and three-torsion and compresses to generators $(-1,0)$ and $(0,1)$, certifying closure. A hidden rank-two relation is retained, duplicate and infinity presentations are compressed, an unresolved coefficient-zero example returns step-limit, and a six-generator output returns width-limit.

SubgroupDivisionClosure proves that closure under m and n implies closure under mn , and under n implies closure under every power n^k . Consequently the final closed subgroup is closed under every multiplier $2^a 3^b$. This statement is unconditional once the subgroup closure premises have been established; it neither states ambient rank nor identifies the full Mordell-Weil group.

A real singular endpoint

Let $K(m)$ denote the complete real elliptic integral with parameter m , so its integrand is $(1-m \sin^2 t)^{-1/2}$. Set $z=1-m$. For positive z less than one, the convergent complementary-modulus expansion is $K(1-z)=\sum_{n=0}^{\infty} a_n z^n$ with $a_n = \frac{1}{16^n} \binom{2n}{n} [2\log(2) - 2(H_{2n} - H_n)]$, with $a_n = \frac{1}{16^n} \binom{2n}{n} [2\log(2) - 2(H_{2n} - H_n)]$. This is DLMF 19.12.1 and 19.12.3 after converting modulus to parameter. The normalization is part of the packet; silently substituting k for m would change its meaning.

The coefficients satisfy $0 < a_n \leq 1$. The constants $d_n = 2\log(2) - 2(H_{2n} - H_n)$ are positive, decrease to zero, and are bounded by 2. For a truncation before $n=N$, its omitted tail is positive and at most $z^{N+1} / (1-z)$. Exact rational logarithm intervals are

computed by the repository's atanh-series machinery. The finite sum and this tail give an enclosure with no floating-point error estimate.

Regularization cancels the common logarithm symbolically rather than subtracting two independently rounded quantities. Since the $n=0$ term is $-\log(z)/2 + \log(4)$, the remaining positive terms are bounded by $z[-\log(z)/2 + 2]/(1-z)$, which tends to zero. Thus $K(1-z) + \log(z)/2$ tends to $\log(4)$, while K itself diverges. At $z=0$ the packet returns the logarithmic coefficient $-1/2$ and an enclosure of the finite part $\log(4)$, and no ordinary finite period interval.

Five worked packets cover the endpoint and four positive rational complements down to 2^{-40} . High-precision mpmath checks are independent numerical validation, not proof of the interval guarantee. The convergent-series identification and regularized limit are classical analytic mathematics with an explicit source, not newly kernel-checked theorems. This restricted real scalar endpoint does not certify arbitrary homology markings, complex approach sectors, higher-genus singular matrices or source-generated resonant Frobenius series.

Polynomial source compilation

PolynomialSourceSemantics defines a typed rational expression language with constants, a variable, addition, multiplication, negation and natural powers. Its compiler produces a native rational polynomial. `compile_correct` proves evaluation preservation at every rational input by structural induction. `equality_correct` proves that a source equality is equivalent to a zero of the compiled difference polynomial. No finite-domain bound is used.

This closes source-to-polynomial semantics for that language. It does not identify the Python parser's AST with the typed language, verify automatic route selection, or establish preservation through every catalogue object, staged service operation and native emitter. Those larger compiler obligations remain distinct from the proved structural theorem.

Global obligations that remain

The named MatveevLB predicate in `AnalyticBridge.lean` is a lower bound for three logarithmic forms over all integer exponents. Generated Mordell classifications explicitly quantify MatveevCase premises. This release neither removes those arguments nor introduces an axiom to replace them. Closing that front requires formal algebraic-number heights, degree and positivity hypotheses, the quantitative logarithmic-form theorem, and proof that each generated constant dominates its actual instance. Checking finitely many exponents cannot establish the universal premise.

A general complete rational Mordell-Weil basis requires a rank upper bound and control of the remaining index at all relevant primes. A complete integral-point enumeration additionally needs an effective global height or exponent bound and its verified reduction to a finite search. The bounded preimage driver does not provide those facts. Arbitrary

primes beyond three, an unbounded terminating saturation procedure, and generic Smith/Hermite presentation reduction are also outside this release.

The handoff names each remaining boundary instead of marking all seven broad fronts closed. The concrete next formal push is to produce native typed staged fibres from every existing two/three packet, extend the already proved literal halving-list and original-model transport interfaces to tripling and composition, and connect the tripling quotient to the actual point law including exceptional points. Validate anchored fibres, root-free fibres, nonsquare lifts, rational nine-torsion and composed empty branches. Global Mordell/Matveev and arbitrary singular endpoints remain larger independent theorem developments.

Reproduction and evidence

Run `make elliptic-bridges` with Lean and Mathlib 4.20.0. It builds the five new modules, audits their declared theorem axioms, checks the generated typed root packet, runs the focused Python tests and reproduces six saturation packets, five endpoint packets and five cold service responses. The theorem audit permits only `propext`, `Classical.choice` and `Quot.sound`; no new axiom, sorryAx or `Lean.ofReduceBool` is accepted.

The validation receipt records the actual focused checks and broader regression run. The complete source archives include every tracked repository path and should be extracted into the same directory. Their manifests identify the published commit. The monograph, source, receipts and exact handoff all travel together; caches and toolchains are excluded.

Analytic reference: NIST Digital Library of Mathematical Functions, <https://dlmf.nist.gov/19.12>, equations 19.12.1 and 19.12.3, accessed 2026-10-06.